



University
of Glasgow

W9-01: Cassandra

COMPSCI4064 (H) and COMPSCI5088 (M)

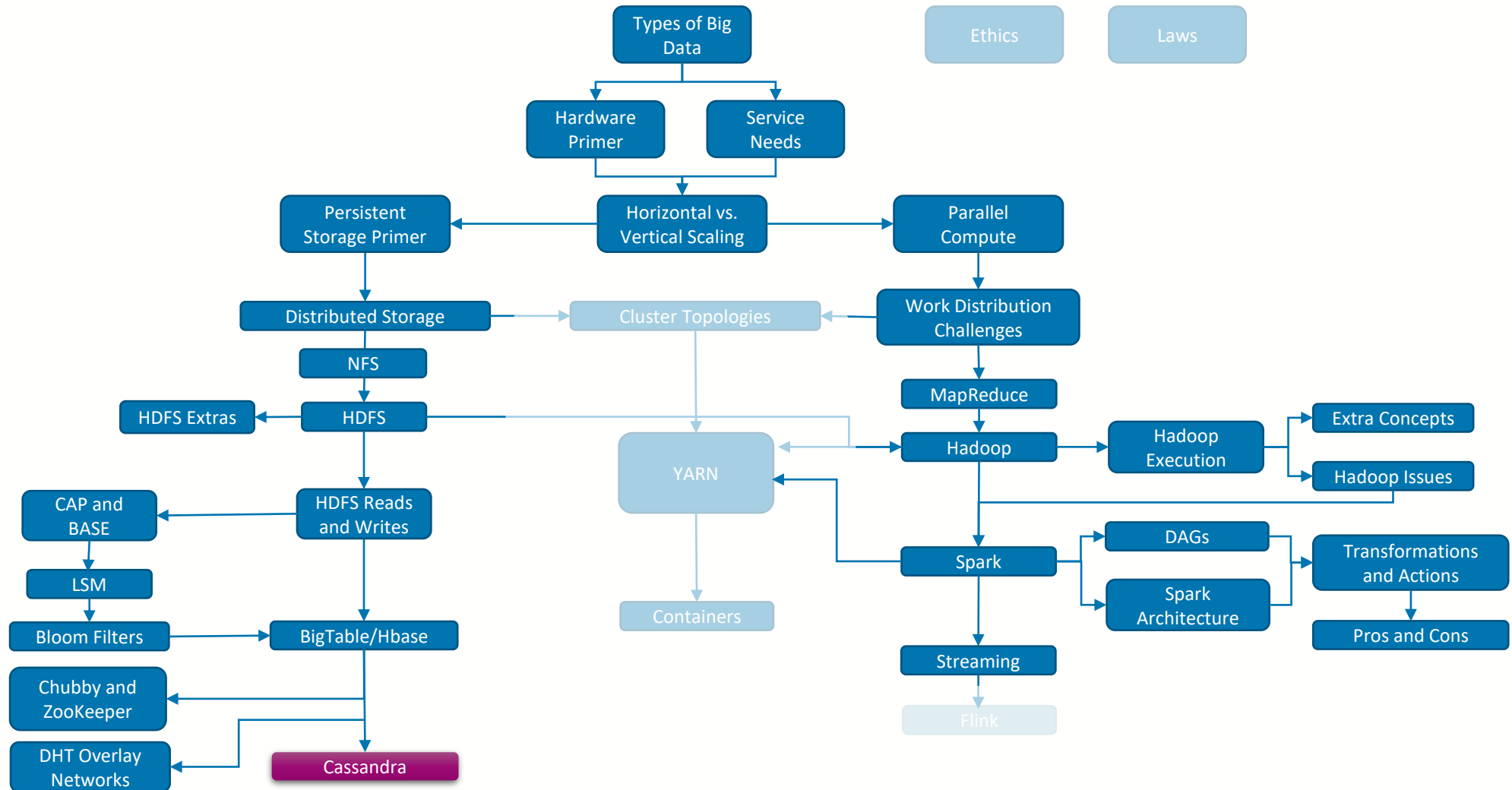
Dr. Richard McCreddie

**WORLD
CHANGING
GLASGOW**

**A WORLD
TOP 100
UNIVERSITY**



Where are we?



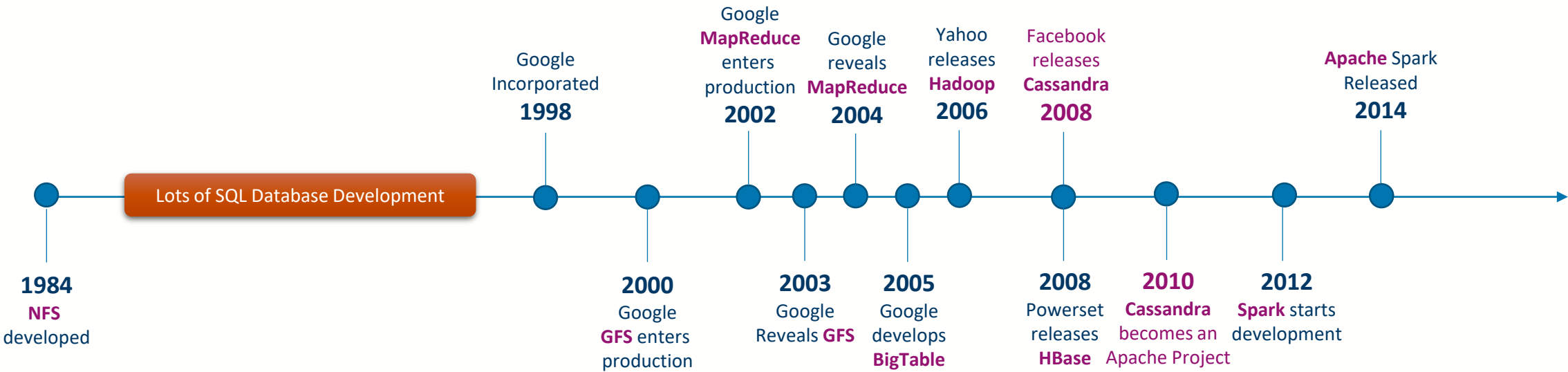


Apache Cassandra

- Cassandra is a **distributed key-value data store** (so its like HBase/BigTable)
 - Indeed, it **inherits Google's BigTable data and storage engine model**
 - But **removes the single-point-of-failure that is master services**
 - Inspired by Amazon's Dynamo distributed storage and replication techniques
- Cassandra is a **peer-to-peer** storage solution
 - ...rather than a master-worker solution like we have seen so far



Technology Timeline





Builds on what we have learned so far...

- Cassandra is focused on **Availability** and **Partition Tolerance** (see CAP)
 - Users can dynamically trade Availability for Constancy
- Cassandra follows **BASE principles**
 - **B**asically **A**vailable
 - **S**oft State
 - **E**ventually consistency
- Cassandra handles reads and write via a **Log Structured Merge Tree**
- Cassandra inherits the **sparse map-based data model** proposed by BigTable/HBase
- Uses a **distributed hash table** design to avoid the need for master services



Cassandra Data Model

- Inherits from BigTable/Hbase
 - It's a distributed **multi-dimensional map** indexed by a key
 - Data can be looked-up by **partition key**, and **column families** can be used to **group data** in the columns
- Key Differences
 - The base data reference is a **<key,column>** rather than a **<key,column qualifier,timestamp>**
 - **No default timestamps**
 - **Column families** can be **multi-tiered**
 - You can have column families of column families

Partition Key	Name	Degree
000001	David	MSc CS
000002	Jing	MSc CS
000003	Xi	MSc DS
000004	John	MSc DS
000005	Ellen	
000006	Yang	
000007	Misbah	MSci Joint
000008	Simona	MSci



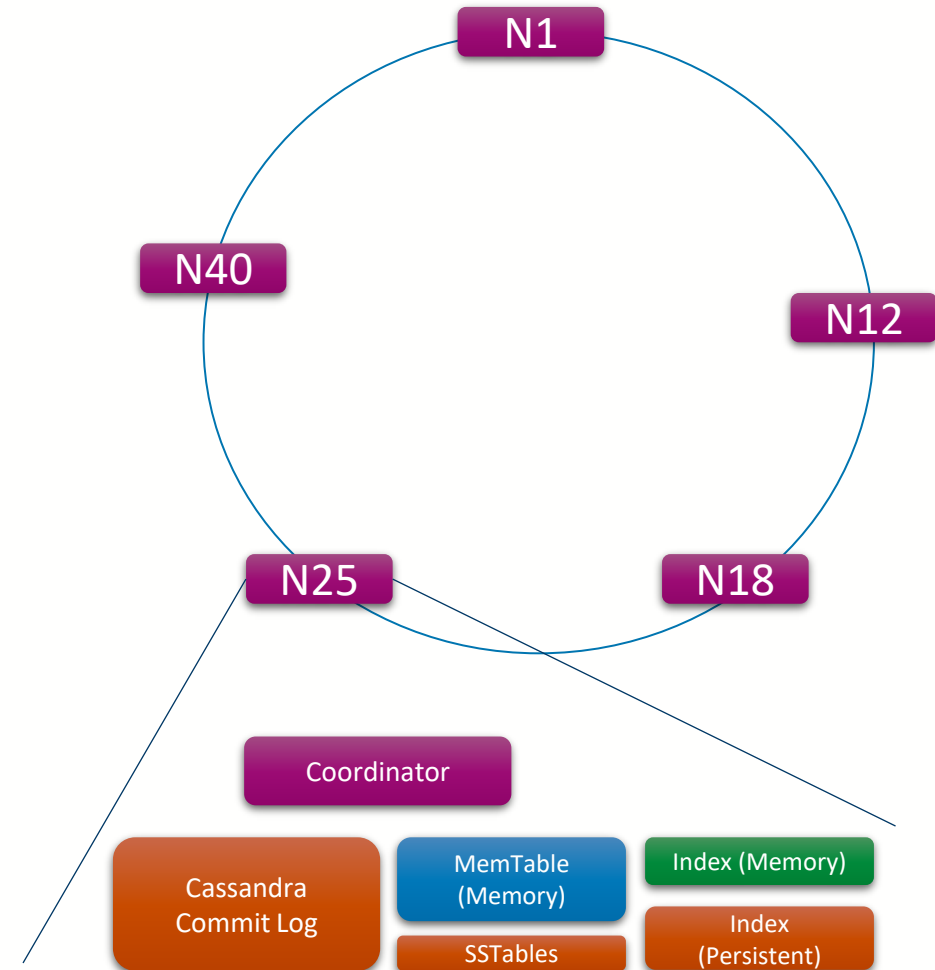
Cassandra API

- Taken from Distributed Hash Tables
 - **insert** (table, key, rowMutation)
 - **get** (table, key, columnName)
 - **delete** (table, key, columnName)
- **Operations** on a given **row** are **atomic** (per replica)
 - Regardless of how many columns are accessed!



Cassandra Architecture

- ... It's a distributed hash table
 - Machines are called **Nodes**
- Within a node it's a **Log Structured Merge Tree**
 - **Cassandra Commit Log**: Is the Write Ahead Log
 - **MemTables**: ...are MemTables
 - **SSTables**: ...are SSTables
 - **Index**: Maps <table,key> pairs to SSTables
- Also includes a **Coordinator**
 - Handles communication and network transfer between nodes



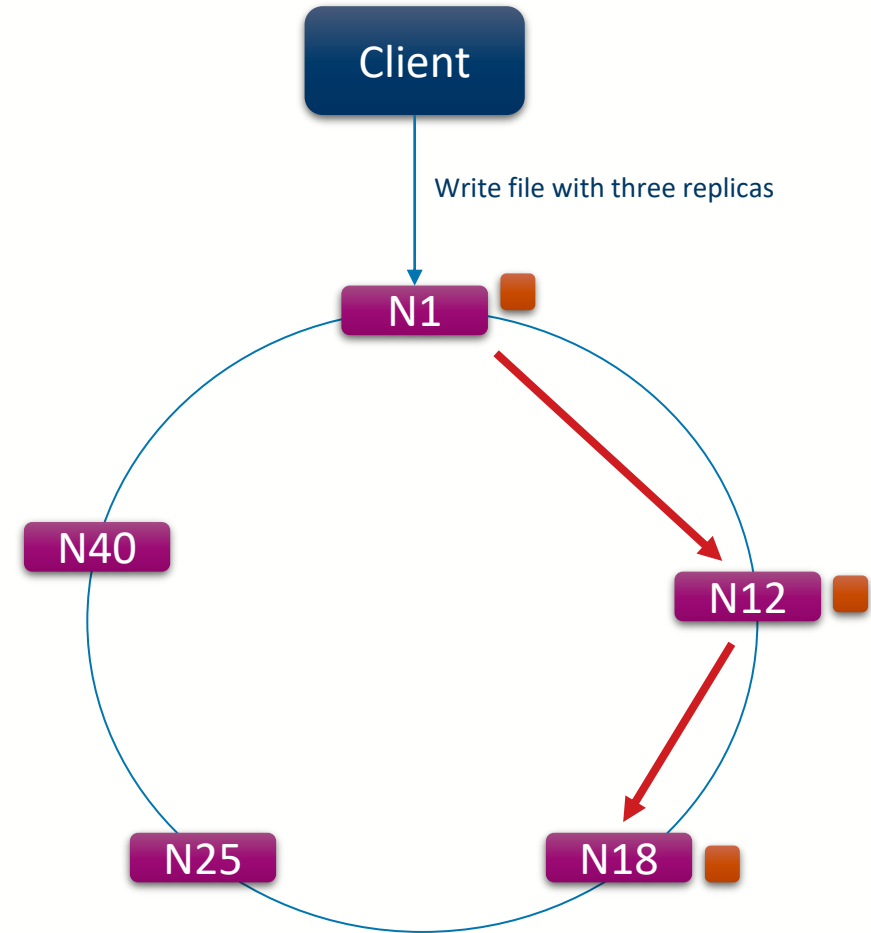
Which node gets the first copy of the data?

- As the system uses a distributed hash table, we apply a **hash function** to the data to generate a **partition key**, which is used to assign data to a node:
- Two main hash options:
 - **RandomPartitioner**: This uses a hash function with a random output in a given range of values (e.g. random ring position)
 - Advantage: Simple to calculate
 - Disadvantage: Does not enable load balancing
 - **OrderPreservingPartitioner**: This uses an **order preserving hashing function**
 - Aside: An order preserving hash function is one where if $x \leq y$ then $\text{hash}(x) \leq \text{hash}(y)$
 - Advantage: Great for doing range queries because data with similar keys will be near each other
 - Disadvantage: Can cause data to get clustered together on a few nodes



Replication Strategy

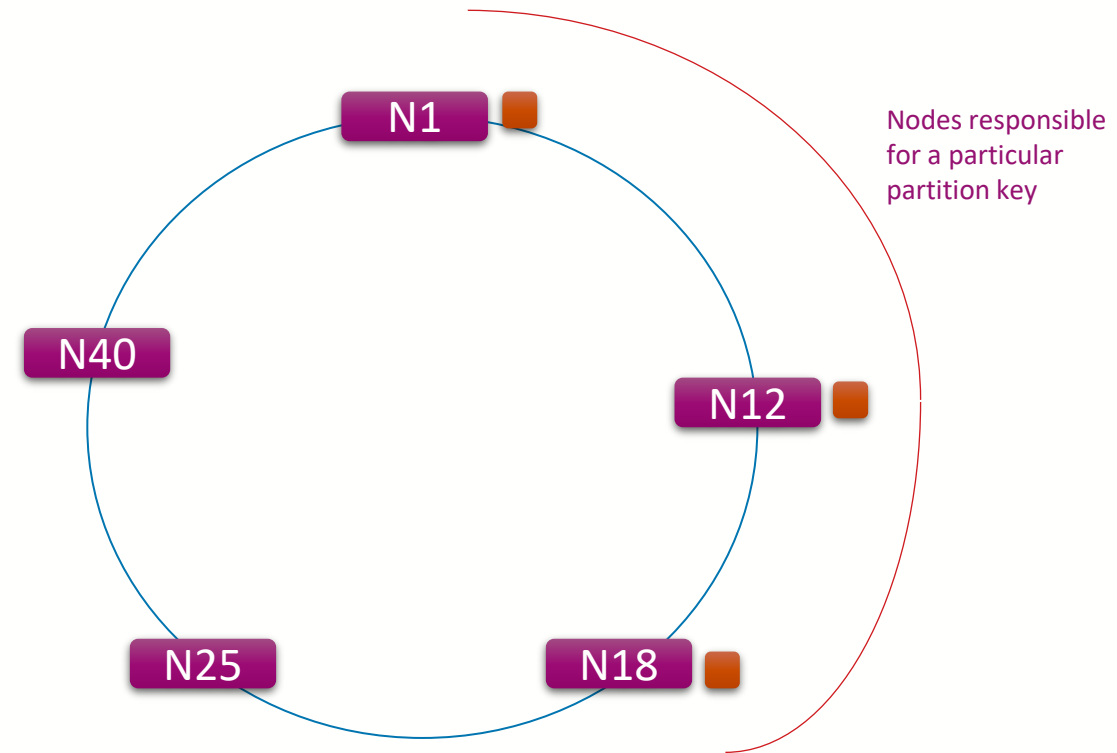
- Very simple once the first node is selected:
 - Just select the **next n nodes** in the ring (clockwise) to hold the replicas





Replication Strategy

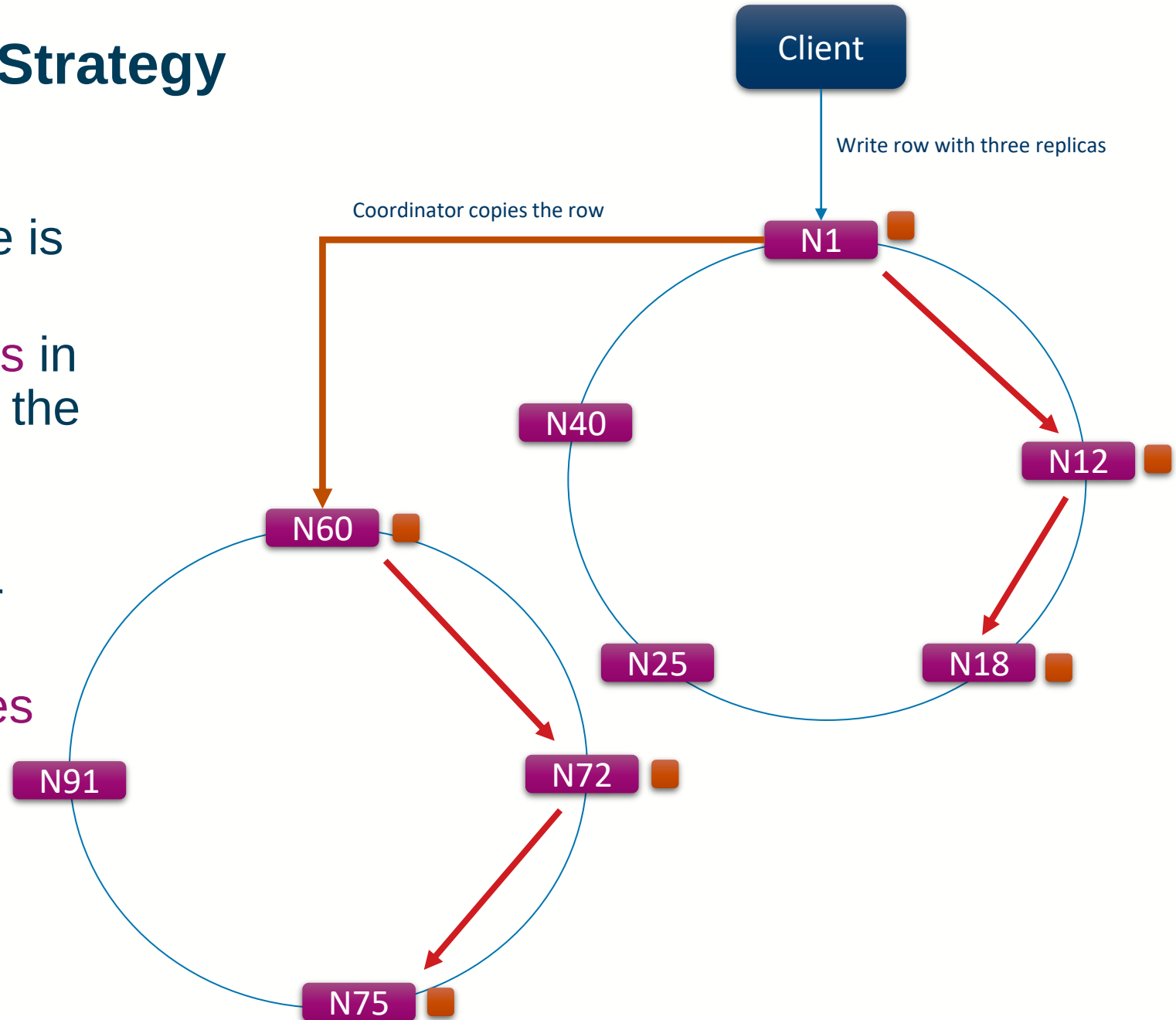
- Very simple once the first node is selected:
 - Just select the **next n nodes** in the ring (clockwise) to hold the replicas
- This means that groups of nodes within an **arc** are responsible for a **single partition key**
 - Nodes within this arc monitor each other to **detect failures** and trigger re-replication





Replication Strategy

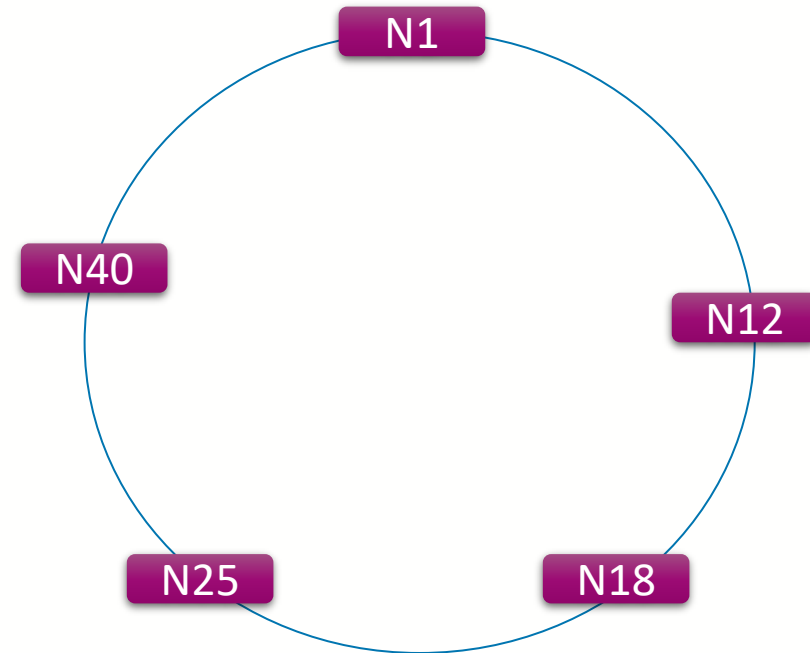
- Very simple once the first node is selected:
 - Just select the **next n nodes** in the ring (clockwise) to hold the replicas
- Alternatively if spread out over multiple data centres, also **replicate across all data centres**
 - More advanced strategies are available, see snitches





Thinking with Arcs

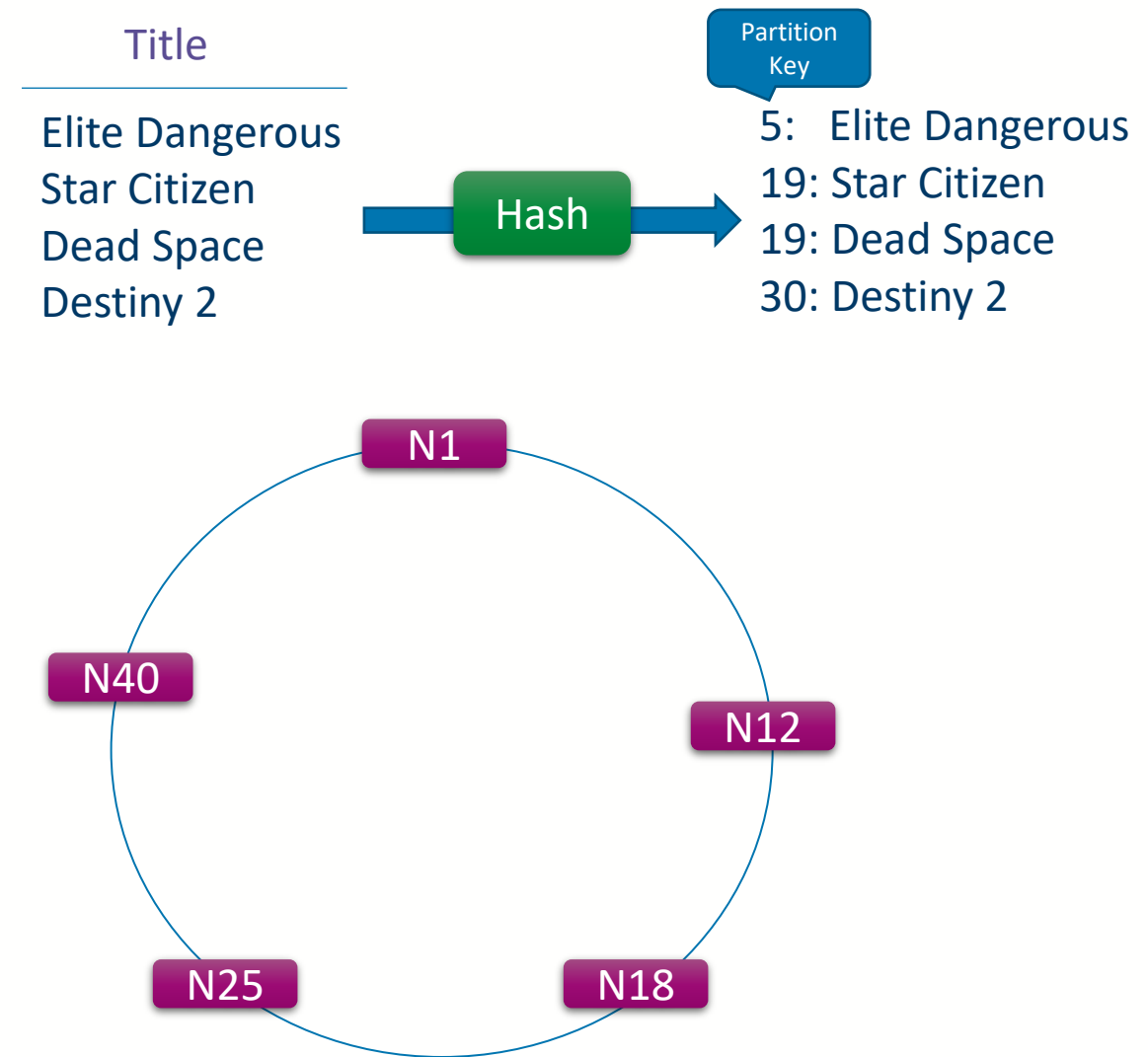
- When thinking about where data ends up within Cassandra it is important to understand the concept of partition keys and arcs
- Consider the example across:
 - We have 5 nodes





Thinking with Arcs

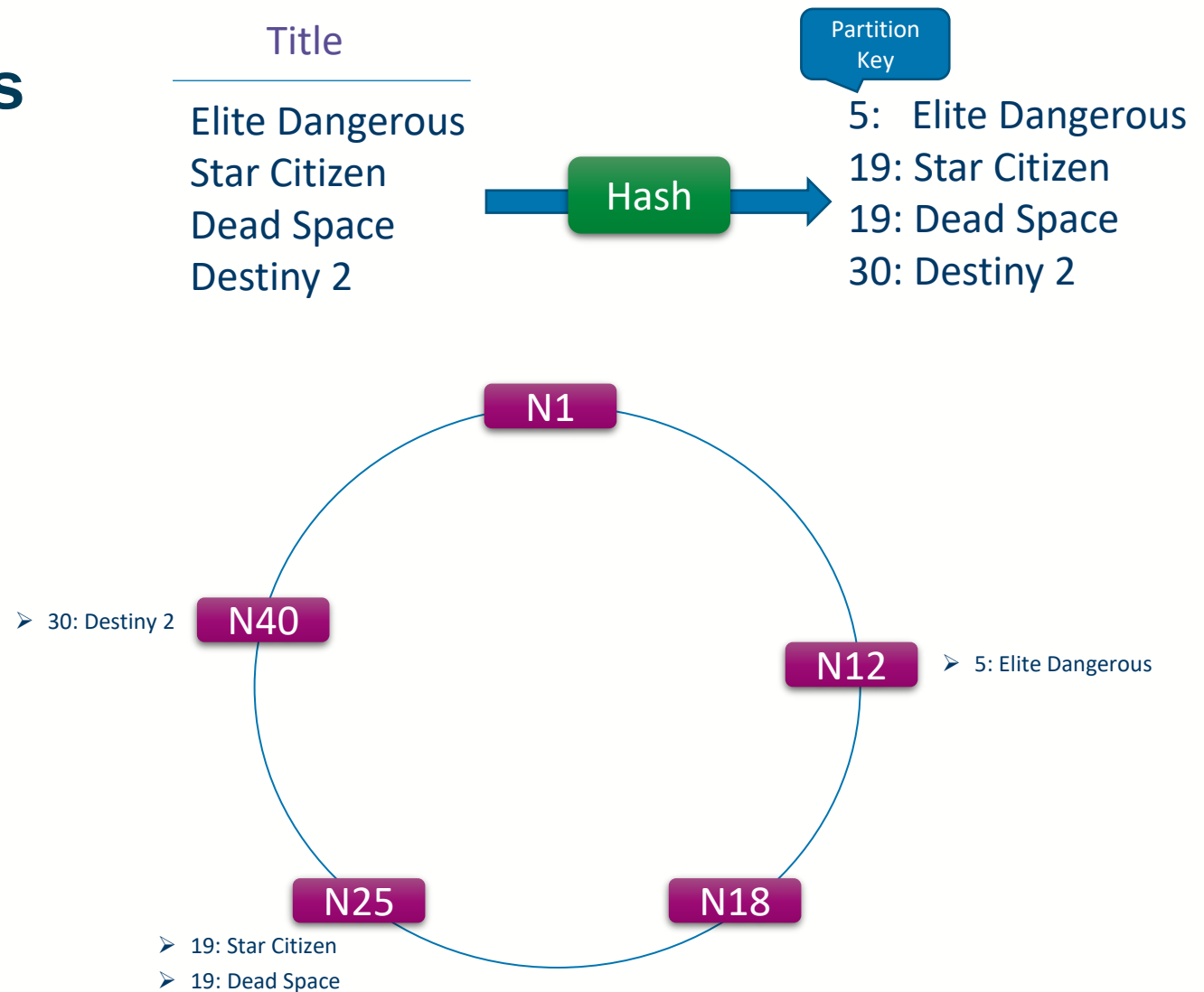
- When thinking about where data ends up within Cassandra it is important to understand the concept of partition keys and arcs
- Consider the example across:
 - We have 5 nodes
 - Lets assume we have 4 rows, two of which result in the same partition key when hashed





Thinking with Arcs

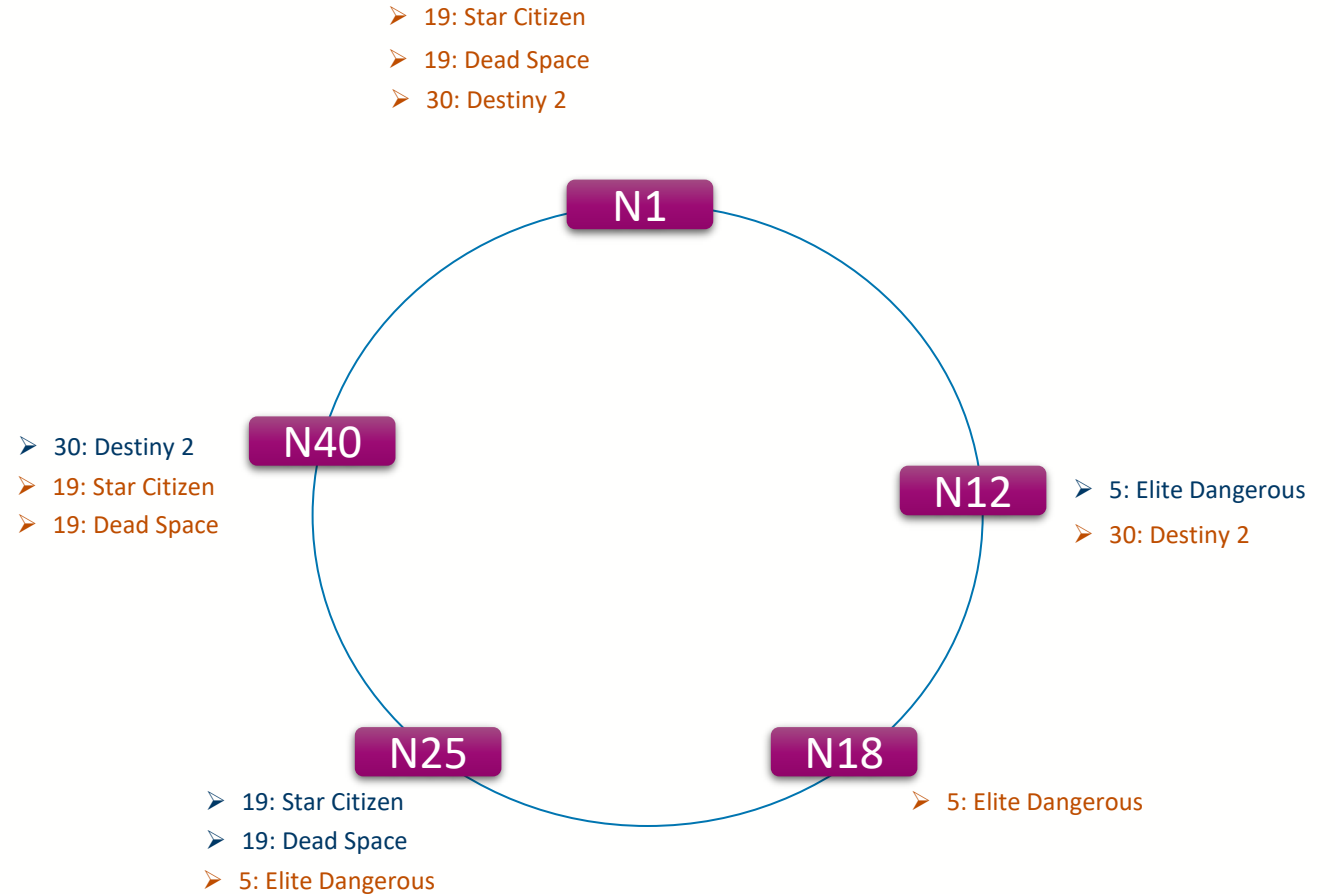
- The different rows will be assigned to nodes based on their partition key
 - If a node with the same number as the key does not exist, select the next highest





Thinking with Arcs

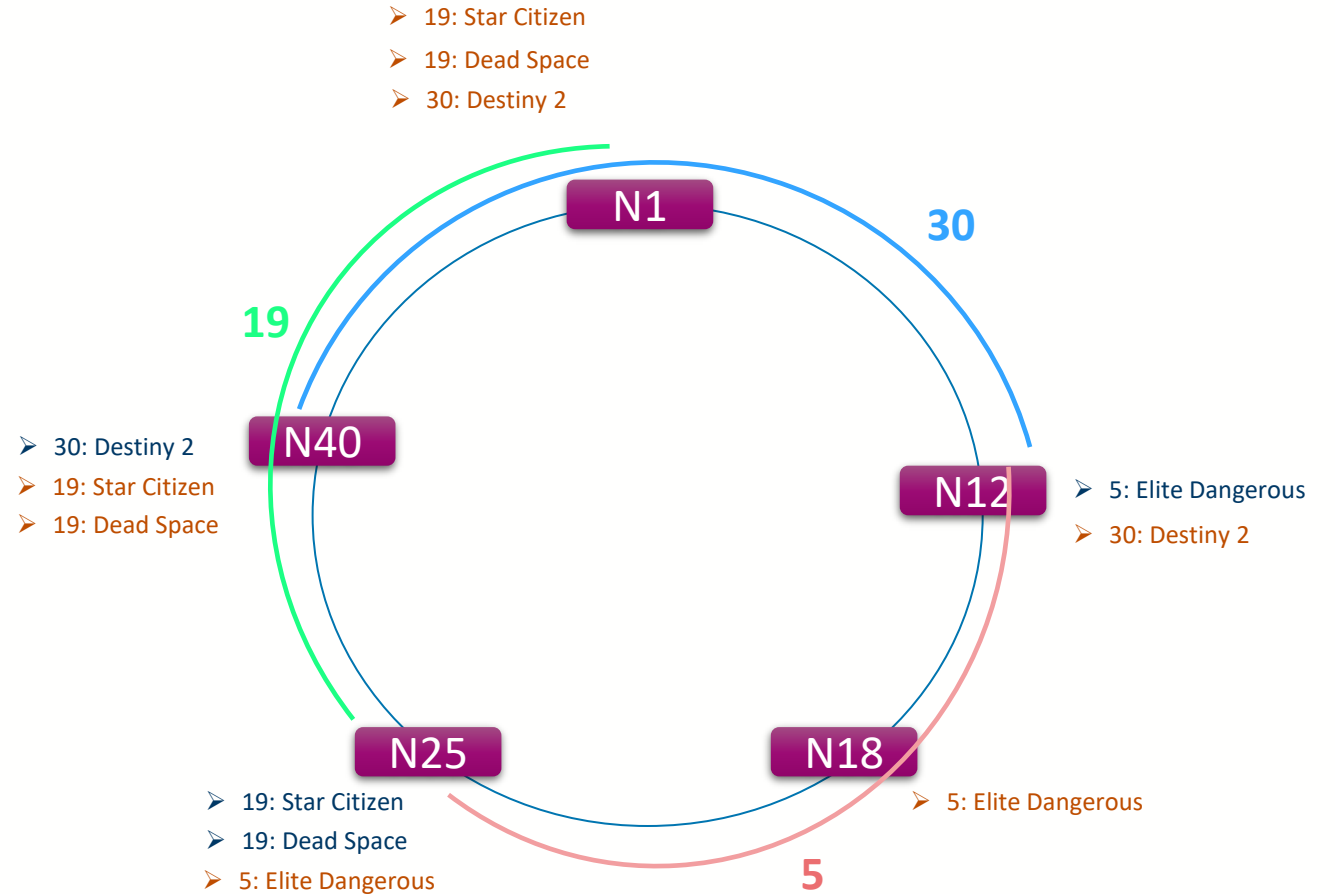
- The different rows will be assigned to nodes based on their partition key
 - If a node with the same number as the key does not exist, select the next highest
- If replication is set to 3, the coordinator on the selected node will copy the data to the next three nodes in the ring





Thinking with Arcs

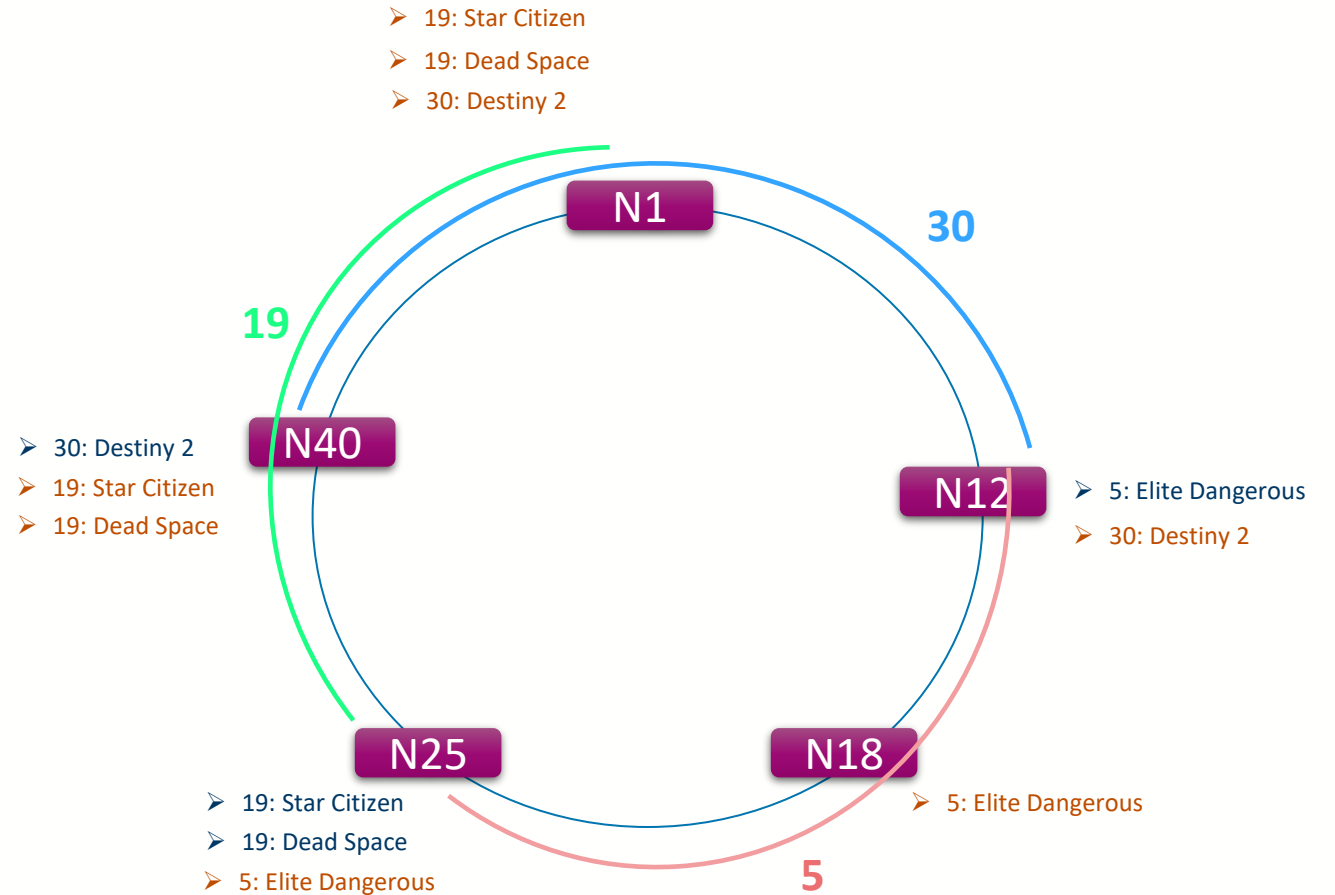
- In this way, we can consider each of the partition keys to have their own **arc**
 - The '5' arc spans N12-25
 - The '19' arc spans N25-N1
 - The '30' arc spans N40-N12





Loosing a Node

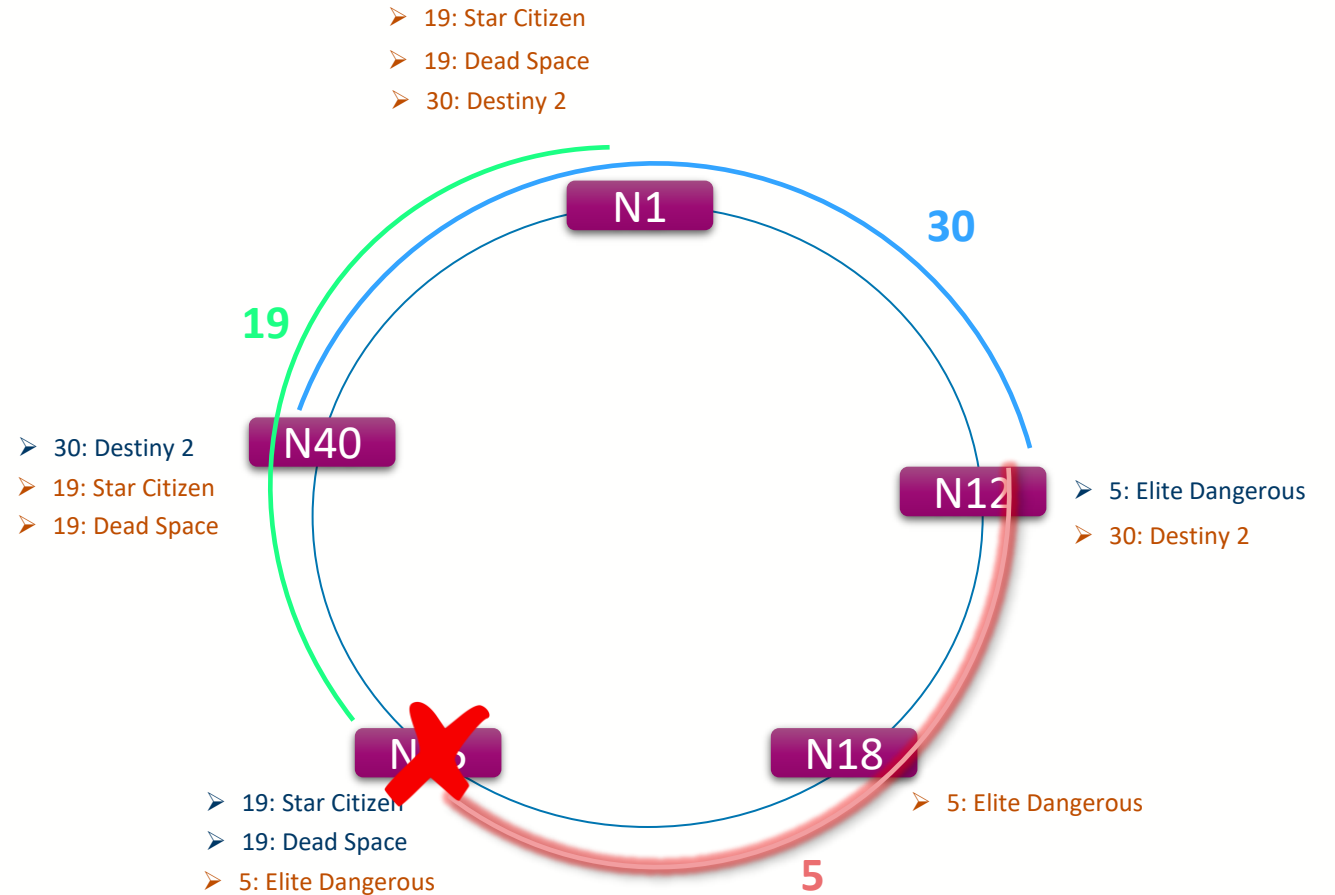
- If a node drops out of the cluster, then the other nodes in the same arc will detect it
 - Extend the arc to the next node





Loosing a Node

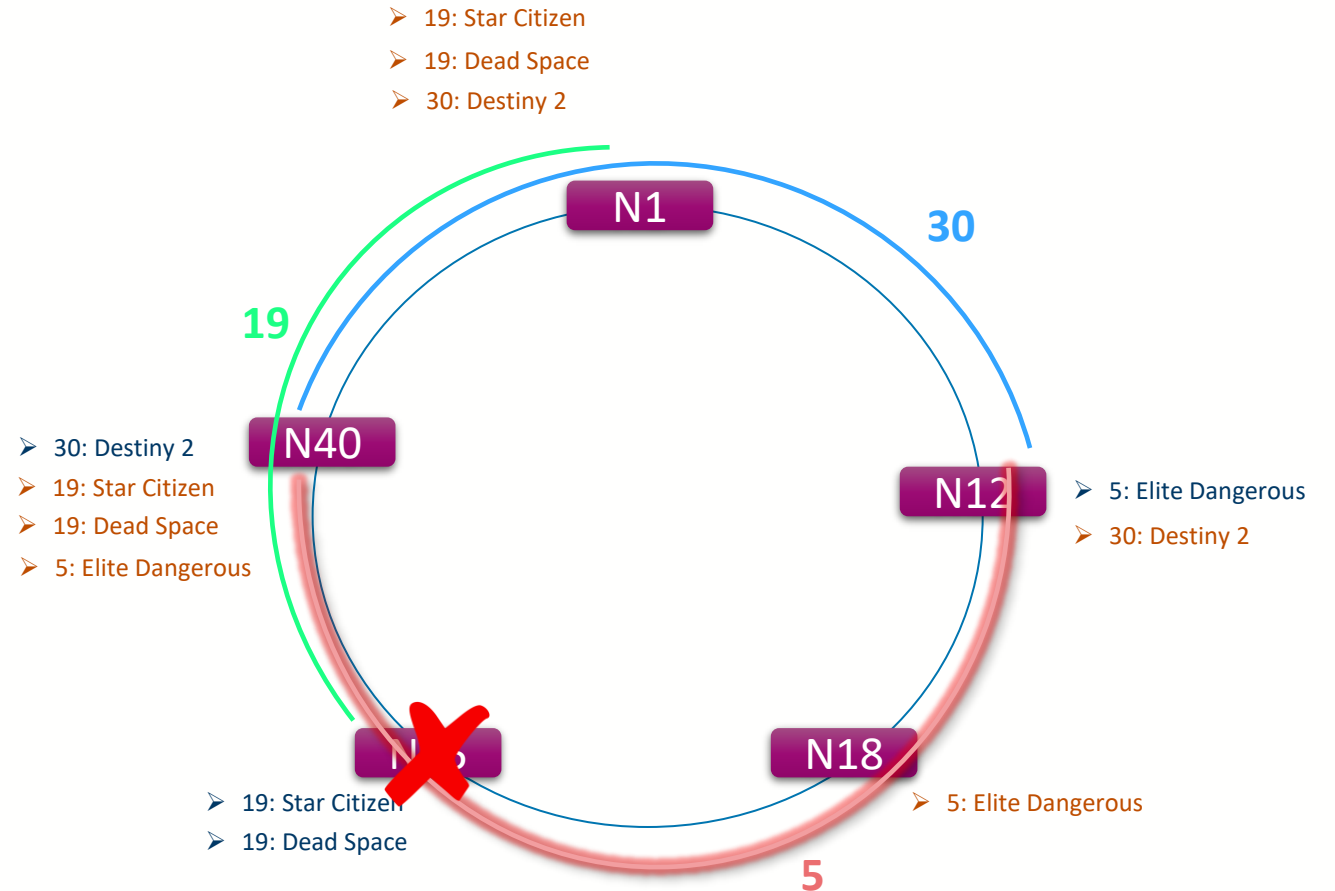
- If a node drops out of the cluster, then the other nodes in the same arc will detect it
 - Extend the arc to the next node





Loosing a Node

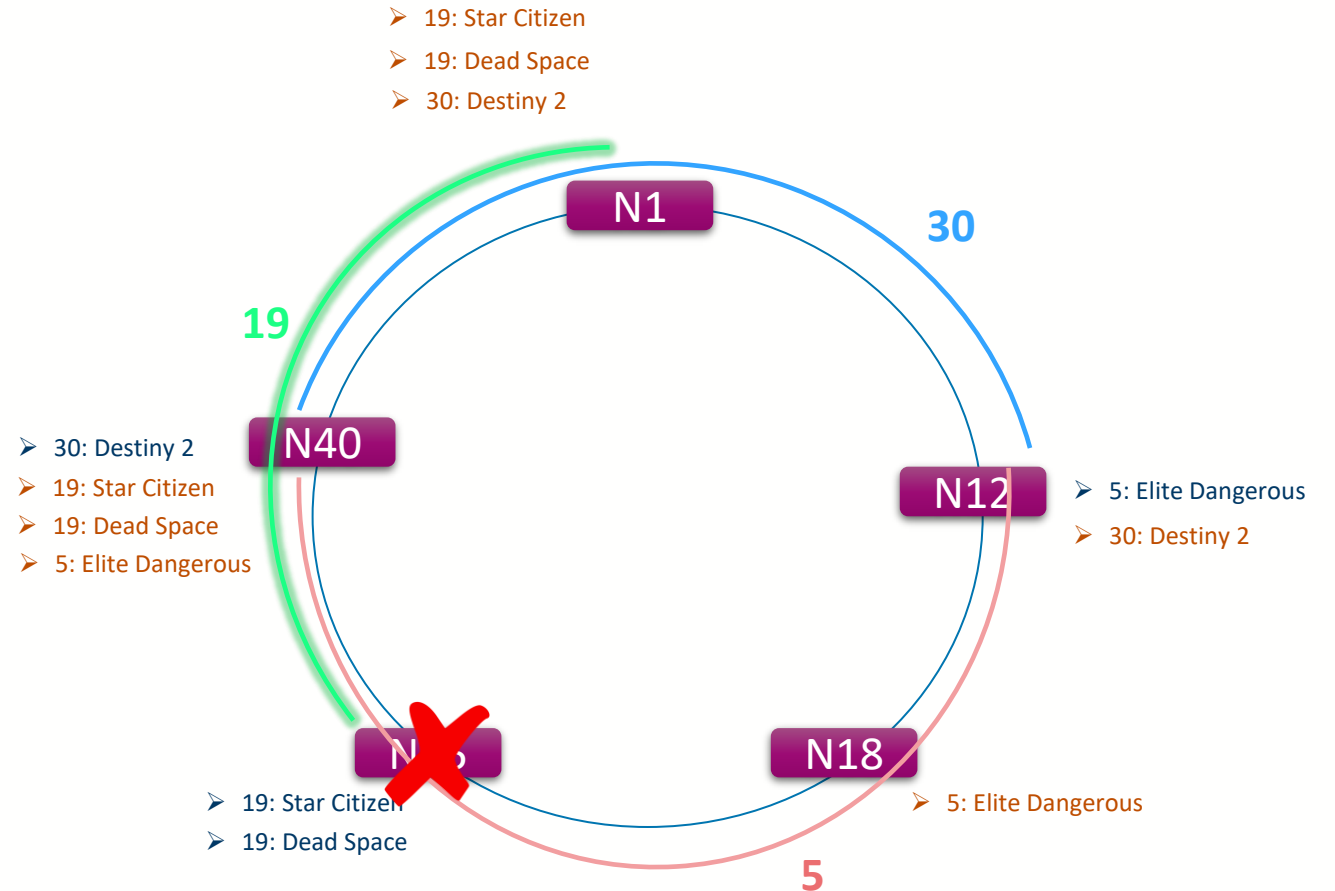
- If a node drops out of the cluster, then the other nodes in the same arc will detect it
 - Extend the arc to the next node





Loosing a Node

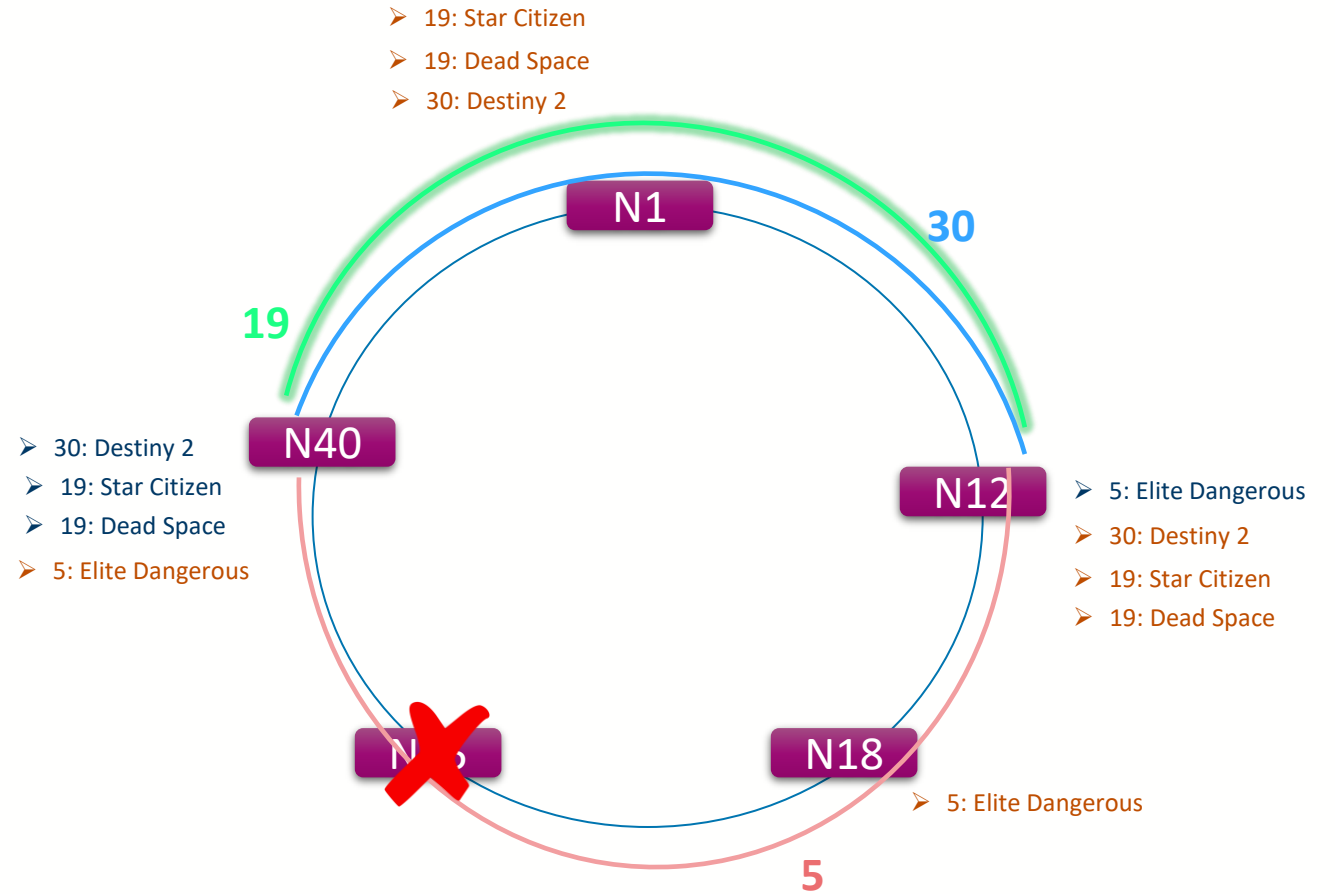
- If a node drops out of the cluster, then the other nodes in the same arc will detect it
 - Extend the arc to the next node





Loosing a Node

- If a node drops out of the cluster, then the other nodes in the same arc will detect it
 - Extend the arc to the next node





University
of Glasgow

Course Coordinator
Dr. Richard McCreadie

Email: richard.mccreadie@glasgow.ac.uk
Room: SAWB 304

#UofGWorldChangers



@UofGlasgow